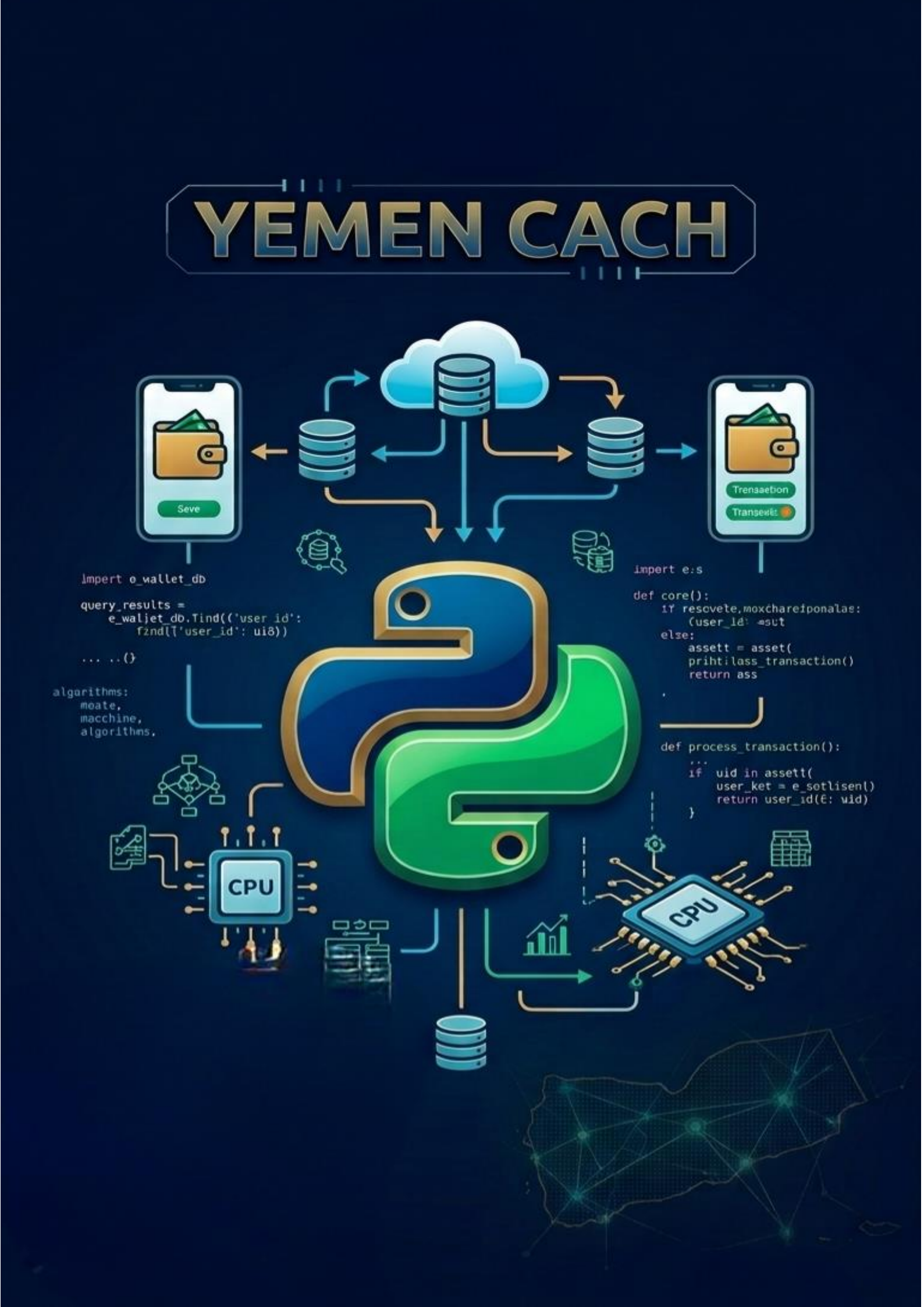

YEMEN CACH
Seve
Trenaaetion
CPU
CPU



الجامعة : اليمنية

الكلية : كلية الهندسة والحاسوب

القسم : الأمن السيبراني

المستوى : الثاني

Project : Object Oriented programming

Language : Python

Database : Oracle

Project's Name : Electronic Wallet System

إشراف :

د/ علي الماوري

إعداد الطالبات :

أصيلة الشاوري

١. فكرة المشروع

نظام محفظة إلكترونية يهدف إلى تسهيل التعاملات الإلكترونية بطريقة مبسطة ومنظمة ويعتمد النظام على قاعدة بيانات مترابطة .

٢. الهدف من المشروع

يهدف المشروع إلى تطبيق مفاهيم البرمجة كائنية التوجة (oop) باستخدام لغة (python) مع ربط البرنامج بقاعدة بيانات (oracle) وتشمل الأهداف التالية :

- تسهيل التعاملات المالية الإلكترونية
- توفير الوقت والجهد في إدارة العمليات المالية
- تنظيم البيانات المالية ومتابعة العمليات
- تقليل الاعتماد على التعامل النقدي
- توفير نظام قابلاً للتطوير مستقبلاً

٣. Classes المستخدمة في المشروع ووظيفة كلاً منها

تم تحويل كل كيان (Entity) في قاعدة البيانات إلى كلاس (class) في python

والجدول التالي يوضح الكلاسات ووظيفة كلاً منها .

#	اسم الكلاس	وظيفته
١	Database	مسؤول عن الاتصال بقاعدة البيانات وتنفيذ الاستعلامات
٢	User	يمثل المستخدم ويضيف (رقم المستخدم ، الاسم ،البريد ، الهاتف ،..)
٣	Wallet	يمثل المحفظة المالية الخاصة بالمستخدم ويضيف (رقم المحفظة ،..)
٤	Currency	يمثل العملات المدعومة في النظام (ريال يمني ، ريال سعودي ، دولار)
٥	Wallet_currecy	يمثل حساب العملة داخل المحفظة ويحدد الرصيد المتاح لكل عملة
٦	Transaction	يمثل العمليات المالية (تحويل ، إيداع ، سحب) مع المبلغ والحالة
٧	Beneficiary	يمثل المستفيد ويضيف (رقم المستفيد ، الاسم ، الهاتف ، الحالة ،..)
٨	Notification	يمثل الإشعارات التي تصل إلى المستخدم عند حدوث حركة مالية
٩	Notification_details	يمثل التفاصيل الإضافية للإشعار (رقم المرجع ، العمولة ، ..)

ملاحظـــة :

تم تنظيم الكود في ملفات منفصلة كالتالي :

- Datebase.py
- user.py
- wallet.py
- currency.py
- wallet_currency.py
- transaction.py
- beneficiary.py
- notification.py
- notification_details.py
- main.py

وذلك لتحقيق التنظيم المطلوب .

٤. توضيح مفاهيم (oop) المستخدمة

أ- التغليف (Encapsulation):

تم تطبيقه من خلال جعل الخصائص خاصة (private) باستخدام الشرطتين (__) ومنع الوصول إليها مباشرة من خارج الكلاس.

تم توفير دوال (Getter) للقراءة ، دوال (setter) للتعديل

مثال من الكود :

```
class User:
    # constructor with Encapsulation
    def __init__(self, first_name, mid_name, last_name, phone,email,password,dob):
        self._first_name=first_name
        self._mid_name=mid_name
        self._last_name=last_name
        self._phone=phone
        self._email = email
        self._password = password
        self._dob = dob
```

```
# get full Name (Getter)

def get_full_name(self):
    return f"{self._first_name} {self._mid_name } {self._last_name}"


# set_email (setter)

def set_email(self,new_email):
    self._email=new_email
```

ب- التجريد (Abstraction):

تم تطبيقة من خلال كلاس (Database) الذي يخفي تفاصيل الاتصال بقاعدة البيانات ويعرض دوال بسيطة للتعامل مع البيانات مثل (cursor.execute)

موضح في الصورة :

```python
# connection with Database of "ew"
import oracledb

class Database:

    def __init__(self):

        oracledb.init_oracle_client(lib_dir=r"C:\oracle\instantclient-basic-windows.x64-19.32.0.0.0dbru\instantclient_19_32")

        self.connection=oracledb.connect(
            user="ew",
            password="ew",
            dsn="localhost:1521/orcl"
        )
        self.cursor=self.connection.cursor()

    def close(self):
        self.connection.close()

```

شرح مختصر لطريقة ربط البرنامج بقاعدة البيانات :

تم ربط برنامج (python) بقاعدة بيانات (oracle) باستخدام مكتبة (oracledb)

الخطوات كالتالي :

- تهيئة عميل أوراكل
- الاتصال بقاعدة البيانات
- إنشاء المؤشر (cursor)
- تنفيذ الاستعلامات مثل استخدام (:1,:2) كمتغيرات
- ومثل استخدام (fetchone) للصف الواحد و (fetchall) للعديد من الصفوف
- حفظ التغييرات (commit) بعد عمليات (insert , update , delete)
- إغلاق الاتصال (close)

٥. شرح أهم وظائف البرنامج

يحتوي البرنامج على قائمة رئيسية (Main Menu) تتيح للمستخدم التفاعل مع النظام

الوظائف الأساسية هي :

#	الوظيفة	الشرح
١	إضافة (Add)	إضافة مستخدم جديد ، محفظة ، مستفيد ، عملية ، إشعار ، عملة ، ...
٢	عرض (View)	عرض جميع المستخدمين ، المحافظ ، العمليات ، الإشعارات ، ...
٣	بحث (Search)	البحث عن المستخدم بالاسم ، البحث عن عملية برقمها ...
٤	تعديل (Update)	تعديل كلمة المرور ، تعديل عملية ، تعديل حالة المحفظة ، ...
٥	حذف (Delete)	حذف مستخدم ، محفظة ، مستفيد ، ...

آلية العمل:

- التحويلات المالية : عند إضافة عملية تحويل ، يقوم ال(Trigger) في قاعدة البيانات بتعديل الأرصدة تلقائياً (خصم من المرسل ، إضافة للمستلم).

- عند الأضافة لمستخدم أو محفظة أو ... يتقوم دالة ال (sequence) في قاعدة البيانات بزيادة الرقم بـ واحد تلقائياً.

- تعديل الرصيد : يتم يدوياً عبر دالة وذلك عند تحويل أو تعديل عملية من حالة (pending or failed) إلى (completed).

- استدعينا مكتبة (getpass) لإخفاء كلمة المرور عند الإدخال.

- استدعينا مكتبة (os) لتنفيذ أمر مسح الشاشة.

- تم عمل كلاس ل (Database) لاستدعائه في كل ملف من الكلاسات بدلاً من إعادة الاتصال في كل ملف.

- تم تشغيل كل الكلاسات في ملف واحد (main.py) وفيه تم استدعاء كل ملف مع الكلاس.

- وذلك بجعل لكل كلاس من الكلاسات دالة في ملف (main.py) لتنفيذ الدوال التي بداخله مثل دوال (Add ,Update,Delete,Search,view).

- تم إنشاء كائن واحد ال (Database) للاتصال بقاعدة البيانات ومن ثم استدعائه في كل دالة ومن ثم إغلاق الاتصال.

٦. صور إنشاء الجداول في قاعدة البيانات (oracle) :

```
-------------------------------------------------------------
create table USERS(
User_id        number              primary key,
First_Name     varchar2(20)        not null,
Mid_Name       varchar2(20) ,
Last_Name      varchar2(20)        not null,
Phone_Number   number(9)           unique not null,
Email          varchar2(30)        unique ,
Password_user  varchar2(20)        not null,
Date_of_Birth  Date                not null,
Created_At     Date                default sysdate not null
);
-------------------------------------------------------------
إنشاء تسلسل لرقم المستخدم
create sequence user_seq
start with 1
increment by 1
nocache;
```

```
create table WALLET(
Wallet_id       number             primary key,
User_id         number             unique ,
Status_wallet   varchar2(10)       check(Status_wallet in('Active','UnActive')),
Created_At      Date               default sysdate not null,
constraint  user_wal_FK foreign key (User_id) references USERS(User_id) on delete cascade
);
-----------------------------------------------------
إنشاء تسلسل لرقم المحفظة
create sequence wallet_seq
start with 1
increment by 1
nocache;
```

```
--=======================================================
create table CURRENCY(
currency_id        number          primary key,
currency_name      varchar2(20),
currency_code      char(3)
);
-------------------------------------------------------------
إنشاء تسلسل لرقم العملة
create sequence currency_seq
start with 1
increment by 1
nocache;
```

```
--=========================================================
create table WALLET_CURRENCY(
wallet_currency_id     number             primary key,
currency_id            number,
Wallet_id              number,
Balance                number(15,3)       check(Balance >= 0),
constraint  wal_id_FK foreign key (Wallet_id) references WALLET(Wallet_id) on delete cascade,
constraint  cur_id_FK foreign key (currency_id) references  CURRENCY(currency_id) on delete cascade
);
-----------------------------------------------------------
إنشاء تسلسل لرقم حساب المحفظة
create sequence wallet_currency_seq
start with 1
increment by 1
nocache;
```

```
create table TRANSACTION(
transaction_id              number          primary key,
sender_wallet_currency_id   number,
receiver_wallet_currency_id number,
transaction_type            varchar2(50)    check(transaction_tybe in('send','receive')),
transaction_status          varchar2(10)    check(transaction_status in('pending','completed','failed')),
Amount                      number(15,3)    check(Amount > 0)    ,
Created_At                  Date            default sysdate not null,
transaction_desc            varchar2(20),
constraint tra_sen_wal_cur_FK foreign key (sender_wallet_currency_id) references WALLET_CURRENCY(wallet_currency_id),
constraint tra_rec_wal_cur_FK foreign key (receiver_wallet_currency_id) references WALLET_CURRENCY(wallet_currency_id)
);
----------------------------------------
إنشاء TRIGGER للخصم من المرسل والاضافة للمستقبل
create  trigger trg_transfer
after insert on TRANSACTION
for each row
when(new.transaction_status='completed')
begin
update WALLET_CURRENCY set Balance=Balance - :new.Amount
where wallet_currency_id = :new.sender_wallet_currency_id;
----
update WALLET_CURRENCY set Balance=Balance + :new.Amount
where wallet_currency_id = :new.receiver_wallet_currency_id;
end;
-----------------------------------------------------------
إنشاء تسلسل لرقم العملية
create sequence transaction_seq
start with 1
increment by 1
nocache;
```

```sql
create table BENEFICIARY(
beneficiary_id          number          primary key,
User_id                 number,
beneficiary_name        varchar2(100)   not null,
Phone_Number            number(9),
Added_date              Date            default sysdate,
beneficiary_status      varchar2(10)    check(beneficiary_status in('Active','UnActive')),
constraint  user_ben_FK foreign key (User_id) references USERS(User_id) on delete cascade
);
-------------------------------------------------------------
إنشاء تسلسل لرقم المستفيد
create sequence beneficiary_seq
start with 1
increment by 1
nocache;
```

```sql
create table NOTIFICATION(
notification_id         number          primary key,
User_id                 number,
Is_read                 varchar2(10)    check(Is_read in('readable','Unreadable')),
Created_At              Date            default sysdate,
massage                 varchar2(50),
constraint  user_not_FK foreign key (User_id) references USERS(User_id) on delete cascade
)
-------------------------------------------------------------
إنشاء تسلسل لرقم الإشعار
create sequence notification_seq
start with 1
increment by 1
nocache;
```

```sql
--=====================================================
create table NOTIFICATION_DETL(
notification_id number primary key,
transaction_id number ,
reference_number number,
commission number default null,
constraint unq_ref_num unique (reference_number),
constraint not_dtl_FK foreign key (notification_id) references NOTIFICATION(notification_id) on delete cascade,
constraint tra_dtl_FK foreign key (transaction_id) references TRANSACTION(transaction_id) on delete cascade
);
-------------------------------------------------------------
إنشاء تسلسل لرقم تفاصيل الإشعار
create sequence notification_detl_seq
start with 1
increment by 1
nocache;
```

٧. صور لكتابة كود الدوال الأساسية في كلاس العمليات

```python
from database import Database

class Transaction:
    # constructure with Encapsulation

    def __init__(self,sender_wallet_currency_id,receiver_wallet_currency_id,
        transaction_type,transaction_status,Amount,transaction_desc):
        self.__sender_wallet_currency_id=sender_wallet_currency_id
        self.__receiver_wallet_currency_id=receiver_wallet_currency_id
        self.__transaction_type=transaction_type
        self.__transaction_status=transaction_status
        self.__Amount=Amount
        self.__transaction_desc=transaction_desc
```

```python
    # Add Transaction

    def add_transaction(self,db):
        try:
            db.cursor.execute(
                """
                insert into transaction(
                transaction_id ,
                sender_wallet_currency_id  ,
                receiver_wallet_currency_id  ,
                transaction_type ,
                transaction_status ,
                Amount ,
                transaction_desc,
                Created_At
                )values(transaction_seq.nextval,:1,:2,:3,:4,:5,:6,sysdate)

            """,(self.__sender_wallet_currency_id,self.__receiver_wallet_currency_id,
                 self.__transaction_type,self.__transaction_status,self.__Amount,
                 self.__transaction_desc))

            db.cursor.execute("select transaction_seq.currval from dual")
            self.__transaction_id=db.cursor.fetchone()
            db.connection.commit()
            print(f"transaction added successfully ...your transaction_id is {self.__transaction_id}")

        except Exception as failed:
            print(f"transaction added failed {failed}")
```

```python
    # view all transactions

    @staticmethod
    def view_all(db):
        db.cursor.execute("select * from transaction order by 1")
        return db.cursor.fetchall()
```

```python
    # view all the information for sender and receiver

    @staticmethod
    def view_all_sc(db):
        db.cursor.execute("""
            SELECT
            t.transaction_id  AS "operation_id",
            us.First_Name AS "sender_name",
            ur.First_Name AS "receiver_name",
            cs.currency_name AS "sender_currency",
            cr.currency_name AS "receiver_currency",
            t.Amount AS "Amount",
            t.transaction_status AS "operation_status",
            t.Created_At As "operation_date",
            wcs.Balance AS "sender_balance",
            wcr.Balance AS "receiver_balance"
            from TRANSACTION t
            --SENDER
            join wallet_currency wcs on t.sender_wallet_currency_id = wcs.wallet_currency_id
            join WALLET ws on wcs.Wallet_id = ws.Wallet_id
            join USERS us on ws.user_id = us.User_id
            join CURRENCY cs on wcs.currency_id = cs.currency_id
            --RECEVIER
            join wallet_currency wcr on t.receiver_wallet_currency_id = wcr.wallet_currency_id
            join WALLET wr on wcr.Wallet_id = wr.Wallet_id
            join USERS ur on wr.user_id = ur.User_id
            join CURRENCY cr on wcr.currency_id = cr.currency_id
            order by t.transaction_id
        """)
        return db.cursor.fetchall()

```

```python
    # search by transaction id

    @staticmethod
    def search_by_transaction_id(db,transaction_id):
        db.cursor.execute("""
            SELECT
            t.transaction_id  AS "operation_id",
            us.First_Name AS "sender_name",
            ur.First_Name AS "receiver_name",
            cs.currency_name AS "sender_currency",
            cr.currency_name AS "receiver_currency",
            t.Amount AS "Amount",
            t.transaction_status AS "operation_status",
            t.Created_At As "operation_date",
            wcs.Balance AS "sender_balance",
            wcr.Balance AS "receiver_balance"
            from TRANSACTION t
            --SENDER
            join wallet_currency wcs on t.sender_wallet_currency_id = wcs.wallet_currency_id
            join WALLET ws on wcs.Wallet_id = ws.Wallet_id
            join USERS us on ws.user_id = us.User_id
            join CURRENCY cs on wcs.currency_id = cs.currency_id
            --RECEVIER
            join wallet_currency wcr on t.receiver_wallet_currency_id = wcr.wallet_currency_id
            join WALLET wr on wcr.Wallet_id = wr.Wallet_id
            join USERS ur on wr.user_id = ur.User_id
            join CURRENCY cr on wcr.currency_id = cr.currency_id
            where t.transaction_id = :1
        """,(transaction_id,))
        return db.cursor.fetchone()

```

```python

    # update transaction status

    @staticmethod
    def update_status(db,new_status,transaction_id):
        try:
            db.cursor.execute("""
                update transaction
                set transaction_status =:1
                where transaction_id=:2
                """,(new_status,transaction_id))
            db.connection.commit()
            print("updated successfully")

        except Exception as failed:
                print(f"updated failed {failed}")

```

```python
# update balance just for transaction_status = (pending or failed) after update transaction_status

@staticmethod
def update_balance_trans(db,transaction_id):
    try:
        db.cursor.execute("""
            update WALLET_CURRENCY set Balance = Balance-(SELECT Amount from TRANSACTION where transaction_id = :1)
            where  wallet_currency_id = (SELECT sender_wallet_currency_id from TRANSACTION where transaction_id = :2)
            """,(transaction_id,transaction_id))

        db.cursor.execute("""
            update WALLET_CURRENCY set Balance = Balance+(SELECT Amount from TRANSACTION where transaction_id = :1)
            where  wallet_currency_id = (SELECT receiver_wallet_currency_id from TRANSACTION where transaction_id = :2)
            """,(transaction_id,transaction_id))

        db.connection.commit()
        print(" balance updated successfully")

    except Exception as failed:
        print(f"updated failed {failed}")
```

صورة لكود دالة الحذف :

```python
    # delete user

    @staticmethod
    def delete_user(db, user_id):
        try:
            db.cursor.execute("delete from users where user_id =:1", (user_id,))
            db.connection.commit()
            print("deleted successfully")
        except Exception as failed:
            print(f"deleted failed {failed}")
```

٨. صور لكود التشغيل في الملف (Main.py)

```python
import os ···

# ------------------------------------
db = Database()
# ------------------------------------

def clear_screen(): ···

# ------------------------------------
def Users_Management(db): ···

# ------------------------------------
def Wallets_Management(db): ···

# ------------------------------------
def Wallet_Currency_Management(db): ···

# ------------------------------------
def Currency_Management(db): ···

# ------------------------------------
def Transactions_Management(db): ···

# ------------------------------------
def Beneficiary_Management(db): ···

# ------------------------------------
def Notification_Management(db): ···

# ------------------------------------
def Notification_details_Management(db): ···

# ------------------------------------------------
while True: ···

# ------------------------------------------------
db.close()
```

```python
import os
import getpass
from database import Database
from user import User
from wallet import Wallet
from wallet_currency import Wallet_currency
from currency import Currency
from transaction import Transaction
from beneficiary import Beneficiary
from notification import Notification
from notification_details import Notification_details

# ------------------------------------
db = Database()
# ------------------------------------

def clear_screen():
    os.system("cls" if os.name == "nt" else "clear")
```

```python
# ------------------------------------------------
while True:
    print("\n" + "=" * 50)
    print("\tYEMEN CACH\t")
    print("=" * 50)
    print("1- Users Management")
    print("2- Wallets Management")
    print("3- Wallet Currency Management")
    print("4- Currency Management")
    print("5- Transactions Management")
    print("6- Beneficiary Management")
    print("7- Notification Management")
    print("8- Notification_details Management")

    print("=" * 50)

    choice = int(input("Choose Number of Management or 0 for Exit : "))

    if choice == 0:
        print("The Program Closed successfully ")
        break
    elif choice == 1:
        Users_Management(db)

    elif choice == 2:
        Wallets_Management(db)

    elif choice == 3:
        Wallet_Currency_Management(db)

    elif choice == 4:
        Currency_Management(db)

    elif choice == 5:
        Transactions_Management(db)

    elif choice == 6:
        Beneficiary_Management(db)

    elif choice == 7:
        Notification_Management(db)

    elif choice == 8:
        Notification_details_Management(db)

    else:
        print("The number that you chose is wrong !!!")
```

٩. صور لتشغيل البرنامج

```
================================================
        YEMEN CACH
================================================
1- Users Management
2- Wallets Management
3- Wallet Currency Management
4- Currency Management
5- Transactions Management
6- Beneficiary Management
7- Notification Management
8- Notification_details Management
================================================
Choose Number of Management or 0 for Exit : 1

-----Users Management-----
----------------------------------------
1- Add User
2- View All the Users
3- search for a user
4- Update data of user
5- Delete the user
0- to back

----------------------------------------
Choose number from Users Management : 1
```

```
 -----Add New User-----
 Enter First Name : Asmaa
 Enter Mid Name :
 Enter Last Name : kamal
 Enter Phone Number : 774111555
 Enter your Email : asmaa@gmail.com
 Enter your password:
 Enter your Date of birth : 1/1/2005
 User added successfully ...your user id is (10,)

 -----Users Management-----
 ----------------------------------------
 1- Add User
 2- View All the Users
 3- search for a user
 4- Update data of user
 5- Delete the user
 0- to back

 ----------------------------------------
 Choose number from Users Management :
```

```
-----All The Users-----
==============================
user id           : 1
Full_Name         : Aseelah Abd_Algawi Al_shawri
Phone Number      : 780000012
Email             : aseelh@gmail.com
Password          : pass7829
Date_of_Birth     : 2006-10-20 00:00:00
Created_At        : 2026-09-18 15:00:27
==============================
==============================
user id           : 2
Full_Name         : zahraa None Al_motaa
Phone Number      : 780010012
Email             : zahraa@gmail.com
Password          : pass79
Date_of_Birth     : 2007-01-01 00:00:00
Created_At        : 2026-09-18 15:00:27
==============================
==============================
user id           : 3
Full_Name         : Eman None Al_wahedi
Phone Number      : 780011012
Email             : eman@gmail.com
Password          : pass7229
Date_of_Birth     : 2007-01-01 00:00:00
Created_At        : 2026-09-18 15:00:27
==============================
==============================
user id           : 4
Full_Name         : manal None mohammed
Phone Number      : 780110012
Email             : manal@gmail.com
Password          : pass7159
Date_of_Birth     : 2006-10-01 00:00:00
Created_At        : 2026-09-18 15:00:27
==============================
==============================
```

```
search by first name for view information user :
Enter First Name : Aseelah
found it
==============================
user id           : 1
Full_Name         : Aseelah Abd_Algawi Al_shawri
Phone Number      : 780000012
Email             : aseelh@gmail.com
==============================

-----Users Management-----
---------------------------------------
1- Add User
2- View All the Users
3- search for a user
4- Update data of user
5- Delete the user
0- to back

---------------------------------------
Choose number from Users Management :
```

```
------Update password of user------
Enter User id : 1
Enter New Password : aseelah12345
updated successfully

-----Users Management-----
----------------------------------------
1- Add User
2- View All the Users
3- search for a user
4- Update data of user
5- Delete the user
0- to back

----------------------------------------
Choose number from Users Management :
```

```
-----Dlete User-----
Enter User id : 10
Are you sure ? y,n : y
deleted successfully

-----Users Management-----
----------------------------------------
1- Add User
2- View All the Users
3- search for a user
4- Update data of user
5- Delete the user
0- to back

----------------------------------------
Choose number from Users Management : 0
```

```
================================================
        YEMEN CACH
================================================
1- Users Management
2- Wallets Management
3- Wallet Currency Management
4- Currency Management
5- Transactions Management
6- Beneficiary Management
7- Notification Management
8- Notification_details Management
================================================
Choose Number of Management or 0 for Exit : 0
The Program Closed successfully 🖐
PS C:\Users\hp\Desktop\projects>
```

١٠. الخاتمة

في ختام هذا المشروع ، تم تطبيق مفاهيم البرمجة كائنية التوجة (oop) باستخدام لغة (python) مع ربط البرنامج بقاعدة البيانات (oracle).

تم تحقيق الأهداف المرجوة من المشروع وهي :

- تصميم نظام محفظة إلكترونية متكامل .
- تطبيق مفاهيم (Encaosulation , Abstarct) .
- ربط البرنامج بقاعدة بيانات حقيقية (oracle) .
- تنفيذ العمليات الأساسية (إضافة ، عرض ، بحث ، تعديل ، تعديل ، حذف) .
- تنظيم الكود في ملفات منفصلة حسب كل كيان .

١١. المراجع

- Python Official Documentition
- Oracle Database Documentition
- الملازم والشرائح الخاصة بالمادة .